

Tool: Query Optimization Framework

Apa: alur baku diagnosa query lambat — EXPLAIN ANALYZE → identifikasi cost driver → hipotesis fix termurah → ukur before/after. **Kapan dipake:** tiap query lambat > target (SOP-SH-03), atau saat mau justifikasi index baru (SOP-SH-05). **Kenapa:** fix berbasis bukti, bukan tebak. Tiap index ada cost di write — jangan asal tambah. **Framework:** Postgres query planner, COBIT DSS06.

ALUR (ikutin urutan)

1. Ukur baseline

EXPLAIN (ANALYZE, BUFFERS) <query>; — **ANALYZE wajib** (jalanin beneran, bukan estimasi). Catat actual time + rows. Pastikan data volume realistic.

2. Baca cost driver (urut prioritas)

Tanda di plan	Artinya	Fix kandidat
Seq Scan di tabel besar	Baca seluruh tabel	Index pada kolom WHERE/JOIN
rows estimasi ≠ aktual jauh	Statistik basi	ANALYZE <table> / naikin statistics target
external merge Disk (sort)	Sort tumpah ke disk	Index buat ORDER BY, atau naikin work_mem
Nested Loop + rows besar	Join order buruk	Index sisi inner, atau rewrite
Banyak query identik (app log)	N+1 dari ORM	Eager load / 1 join
Heap Fetches tinggi (index-only)	Index gak covering	INCLUDE kolom → covering index

3. Hipotesis fix — TERMURAH dulu

- Rewrite query (hapus SELECT *, hapus fungsi di WHERE yang matiin index) — gratis
- Refresh statistik (ANALYZE) — murah
- Tambah index (proven, SOP-SH-05) — ada write cost
- Denormalisasi / materialized view — mahal, terakhir

4. Ukur after

EXPLAIN ANALYZE lagi. **Bukti = before vs after.** Verifikasi index benar dipakai (bukan masih seq scan).

CONTOH TERISI (Proyek-Contoh — "list RSVP per undangan" lambat)

QUERY:

```
SELECT * FROM rsvp WHERE undangan_id = $1 AND deleted_at IS NULL ORDER BY created_at DESC;
```

=== BEFORE ===

EXPLAIN (ANALYZE, BUFFERS):

Seq Scan on rsvp (cost=0..980 rows=120) (actual time=0.3..14.2 rows=118 loops=1)

Filter: (undangan_id = \$1 AND deleted_at IS NULL)

Rows Removed by Filter: 48000 <-- baca 48rb buat ambil 118

Planning: 0.1ms Execution: 14.2ms <-- > target 30ms? mendekati, dan naik terus seiring data

Cost driver: SEQ SCAN – baca seluruh tabel rsvp buat 1 undangan.

=== FIX (termurah yang works) ===

- Rewrite: ganti SELECT * jadi kolom eksplisit (hindari coupling + heap fetch berlebih)

- Index: partial composite, cover filter + sort

```
CREATE INDEX CONCURRENTLY idx_rsvp_undangan_active
```

```
ON rsvp (undangan_id, created_at DESC) WHERE deleted_at IS NULL;
```

=== AFTER ===

Index Scan using idx_rsvp_undangan_active on rsvp (actual time=0.02..0.4 rows=118)

Execution: 0.5ms <-- dari 14.2ms ke 0.5ms, dan FLAT walau data naik

Verdict: index proven dipakai planner, latency turun ~28x, no redundant (gak ada index undangan_id)
Deploy CONCURRENTLY (no lock) via @levi. Logged sebagai bukti (kontrol D5).

Catatan: partial `WHERE deleted_at IS NULL` bikin index lebih kecil (cuma baris aktif) + langsung align sama scope baca normal (Data SACRED soft-delete). `created_at DESC` di index = sort gratis, no merge-sort. Composite urutan `(undangan_id, created_at)` = left-most prefix `undangan_id` juga kepaake buat query lain.